

Trabalho Prático Individual I

Estruturas em Árvores Avançadas

*Modelagem, Implementação e Análise Comparativa de
Estruturas em Árvore Especializadas*

Aluno(a): [SEU NOME COMPLETO]

Matrícula: [SUA MATRÍCULA]

Disciplina: Estrutura de Dados — 5º semestre

Professor(a): [NOME DO PROFESSOR]

Data de entrega: 19 de setembro de 2026

Repositório com o código-fonte completo: [\[LINK DO GITHUB\]](#)

Sumário

Resumo.....	3
1. Introdução e Fundamentação Teórica.....	3
2. Projeto e Implementação das Estruturas.....	4
3. Demonstração e Rastreamento Visual.....	5
4. Análise de Complexidade e Comparação Teórica.....	7
5. Metodologia Experimental e Resultados.....	8
6. Aplicações, Análise Crítica e Discussão.....	10
7. Conclusão.....	11
Referências.....	11

Resumo

Este trabalho apresenta a modelagem, a implementação e a análise comparativa de cinco estruturas de dados em árvore especializadas: **Trie**, **Árvore Patricia (Radix Tree compacta)**, **Árvore Splay**, **Árvore Treap** e **KD-Tree**. Todas foram implementadas em C++ com as operações de inserção, busca e remoção, além de operações específicas de cada estrutura. Para cada uma são apresentados o rastreamento visual de seus estados, a análise de complexidade assintótica (casos melhor, médio e pior) e experimentos computacionais que confrontam o comportamento teórico com o observado na prática. Os resultados evidenciam que não existe uma estrutura universalmente superior: cada uma é vantajosa em um contexto específico — strings e prefixos (Trie/Patricia), acessos com localidade temporal (Splay), balanceamento simples e probabilístico (Treap) e busca espacial multidimensional (KD-Tree).

Palavras-chave: estruturas de dados; árvores de busca; Trie; Patricia; Splay; Treap; KD-Tree; análise de complexidade.

1. Introdução e Fundamentação Teórica

As **estruturas de dados hierárquicas** (árvores) são fundamentais na organização e na recuperação eficiente de informações. Diferentemente de estruturas lineares, uma árvore permite, em geral, reduzir o custo de busca de $O(n)$ para $O(\log n)$, desde que sua altura seja mantida próxima do logaritmo do número de elementos. As Árvores Binárias de Busca (BST) exploram essa ideia, mas, quando os dados são inseridos em ordem desfavorável, degeneram para uma lista encadeada, com altura $O(n)$. As Árvores AVL resolvem esse problema garantindo balanceamento por meio de rotações e do controle do fator de balanceamento, ao custo de manter informação extra em cada nó.

Motivação. Apesar de eficientes, BST e AVL assumem que as chaves são simples valores comparáveis e que todos os acessos são igualmente prováveis. Muitos problemas reais fogem dessas hipóteses: buscas por **prefixos de strings**, padrões de acesso com **localidade temporal** (poucas chaves acessadas com muita frequência), necessidade de uma implementação **simples e balanceada em média**, ou consultas em **espaços multidimensionais**. As cinco estruturas estudadas surgem exatamente para atender a essas necessidades, cada uma adotando uma estratégia de organização diferente.

1.1. Conceitos fundamentais de cada estrutura

Trie (árvore de prefixos). Árvore n -ária em que cada aresta representa um caractere; uma chave é o caminho da raiz até um nó marcado como fim de palavra. Prefixos comuns compartilham o mesmo caminho. Invariante: o caminho da raiz a um nó descreve exatamente o prefixo acumulado. Anatomia do nó: um vetor/mapa de filhos (indexado por caractere) e um marcador booleano de fim de palavra. Resolve problemas de busca por prefixos e conjuntos de strings.

Árvore Patricia (Radix Tree compacta). É uma Trie comprimida: cadeias de nós com um único filho são colapsadas, de modo que cada aresta guarda uma **substring** em vez de um único caractere. Invariante: nenhum nó interno (exceto a raiz) possui um único filho sem também ser fim de chave. A anatomia do nó acrescenta o rótulo da aresta (substring). As operações usam **divisão (split)** de arestas na inserção e **fusão (merge)** na remoção. Reduz drasticamente o número de nós em relação à Trie.

Árvore Splay. BST autoajustável: a cada acesso, o nó tocado é levado à raiz por rotações (operação splay, nos casos zig, zig-zig e zig-zag). Não armazena altura nem fator de balanceamento; o equilíbrio é apenas **amortizado**. Assim, elementos acessados recentemente ficam próximos da raiz, favorecendo padrões com localidade.

Árvore Treap. Combina duas propriedades: é uma BST pelas **chaves** e um max-heap pelas **prioridades**, sorteadas aleatoriamente na inserção. Como as prioridades são aleatórias, a forma da árvore equivale à de uma BST construída em ordem aleatória, resultando em altura esperada $O(\log n)$ sem regras de balanceamento complexas. O equilíbrio entre as duas propriedades é mantido por rotações simples.

KD-Tree (k-dimensional, aqui $k = 2$). Organiza pontos no espaço, alternando a dimensão de corte a cada nível (nível par corta em x; nível ímpar corta em y). O plano é sucessivamente particionado em retângulos, o que permite consultas espaciais eficientes, como a busca do vizinho mais próximo, aproveitando a poda de regiões que não podem conter a resposta.

1.2. Diferenças conceituais em relação a BST e AVL

- **Tipo de chave:** BST/AVL/Splay/Treap comparam chaves escalares; Trie/Patricia operam sobre a estrutura interna das strings; KD-Tree compara coordenadas por dimensão.
- **Estratégia de balanceamento:** AVL usa balanceamento estrito por altura; Splay usa reorganização por acesso (amortizado); Treap usa aleatoriedade (probabilístico); Trie/Patricia não precisam balancear, pois a altura depende do comprimento das chaves, não de n .
- **Custo em função de n :** em Trie/Patricia as operações são $O(m)$ e independem de n ; nas demais, dependem da altura da árvore.

2. Projeto e Implementação das Estruturas

As cinco estruturas foram implementadas em **C++17**, cada uma em um cabeçalho independente (header-only) na pasta `include/`. As entradas pseudoaleatórias usam semente fixa (42), garantindo **reprodutibilidade**. A seguir descrevem-se as principais decisões de projeto; o código-fonte completo está no repositório indicado na capa.

2.1. Trie e Patricia

Na **Trie**, cada nó mantém um `std::map<char, No*>` (mapa ordenado, para saída alfabética) e um booleano `fimDePalavra`. A inserção percorre a chave criando nós ausentes; a busca segue o caminho e verifica o marcador final; a remoção desmarca o fim de palavra e, ao retornar da recursão, apaga nós que ficaram sem filhos e sem marcação. A decisão-chave é armazenar o marcador de fim no próprio nó, permitindo que uma palavra seja prefixo de outra.

A **Patricia** acrescenta a cada nó o rótulo da aresta que vem do pai (a substring). A inserção calcula o maior prefixo comum entre a chave e o rótulo da aresta atual; quando o prefixo é apenas parcial, a aresta é **dividida** criando um nó intermediário (split). A remoção, além de apagar, pode **fundir** um nó que ficou com um único filho ao seu descendente (merge), concatenando os rótulos e mantendo a árvore compacta. Essa manutenção do invariante de compactação é a principal diferença de projeto em relação à Trie.

2.2. Splay e Treap

A **Splay** usa ponteiros de pai para viabilizar o **splay** ascendente. Inserção e busca são BST comuns seguidas de splay do nó acessado; a remoção dá splay na chave, remove a raiz e

junta as duas subárvores (leva o máximo da subárvore esquerda à raiz e anexa a direita). Os três casos de rotação (zig, zig-zig, zig-zag) estão implementados explicitamente.

A **Treap** é implementada de forma recursiva: na inserção, após descer para o filho, se a prioridade do filho for maior que a do pai, aplica-se uma rotação para fazê-lo subir, restaurando a propriedade de max-heap. Na remoção, o nó a excluir é rotacionado para baixo (sempre subindo o filho de maior prioridade) até virar folha. As prioridades são geradas por um `std::mt19937` com semente fixa.

2.3. KD-Tree

Cada nó guarda um ponto (x, y) e a dimensão de corte é determinada pela profundidade ($\text{prof} \% 2$). A inserção e a busca exata comparam apenas a coordenada da dimensão do nível. A remoção emprega a técnica clássica: ao remover um nó interno, ele é substituído pelo **mínimo, na dimensão de corte, da subárvore direita** (função `findMin`); se não houver subárvore direita, usa-se a esquerda, que passa a ser a direita. A busca do **vizinho mais próximo** desce primeiro para o lado que contém o alvo e só visita o outro lado se a distância ao plano de corte for menor que a melhor distância já encontrada (poda).

Trechos essenciais. Todas as estruturas expõem um método de exportação da topologia (formato textual simples) consumido por scripts Python que geram as figuras deste relatório, evitando reproduzir o código-fonte integral aqui.

3. Demonstração e Rastreamento Visual

Para cada estrutura são exibidos três estados: (1) inicial, após um conjunto de inserções; (2) intermediário, evidenciando uma operação relevante (bifurcação, split, rotação ou particionamento); e (3) após uma remoção/reorganização. Legenda das figuras de Trie/Patricia: nós verdes com * indicam fim de chave; nós cinza são internos; os rótulos em vermelho sobre as arestas são os caracteres/substrings.

3.1. Trie

Inserção de **casa**, **caso** (compartilham o prefixo **cas**); em seguida **carro** cria uma bifurcação em **ca**; por fim remove-se **caso**.

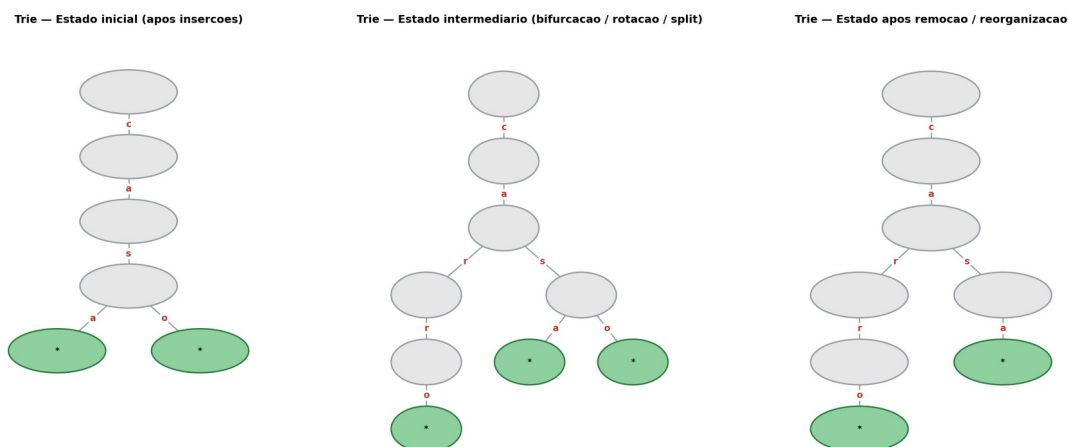
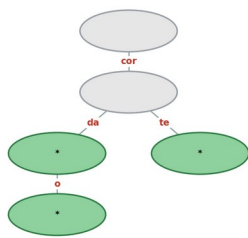


Figura 1 — Trie: estado inicial, bifurcação após inserir "carro" e estado após remover "caso".

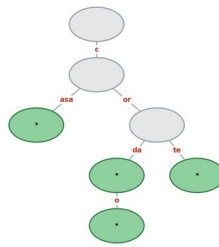
3.2. Árvore Patricia

Inserção de **corda**, **cordao**, **corte**; depois **casa** provoca um split do prefixo em **C**; por fim remove-se **corte**. Observe como as arestas guardam substrings inteiras, reduzindo o número de nós em relação à Trie.

Patricia (Radix compacta) — Estado inicial (apos insercoes)



Patricia (Radix compacta) — Estado intermediario (bifurcacao / rotacao / split)



Patricia (Radix compacta) — Estado apos remocao / reorganizacao

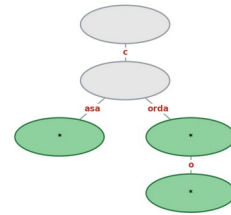
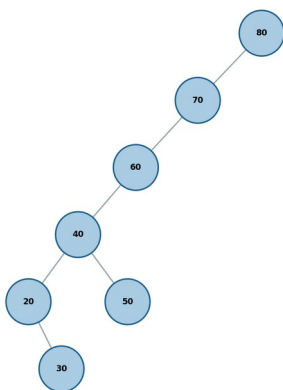


Figura 2 — Patricia: compactação por substrings, split ao inserir "casa" e recompactação após remoção.

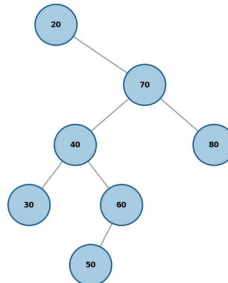
3.3. Árvore Splay

Após inserir 50 30 70 20 40 60 80, uma busca por 20 leva o nó acessado à raiz por rotações (estado intermediário); em seguida remove-se 20. A demonstração evidencia o autoajuste característico da Splay.

Arvore Splay — Estado inicial (apos insercoes)



Arvore Splay — Estado intermediario (bifurcacao / rotacao / split)



Arvore Splay — Estado apos remocao / reorganizacao

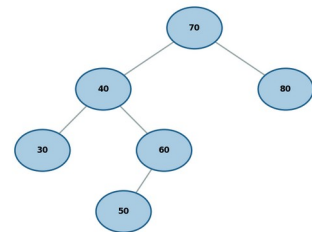
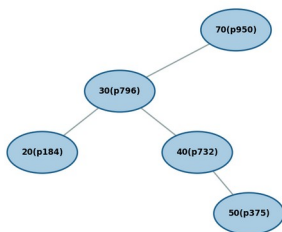


Figura 3 — Splay: estado inicial, splay de 20 até a raiz após a busca e estado após remover 20.

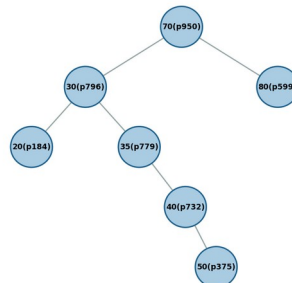
3.4. Árvore Treap

Cada nó mostra chave(pPrioridade); verifica-se que as chaves respeitam a BST e as prioridades respeitam o max-heap (a prioridade do pai é sempre maior que a dos filhos). A inserção de 35 e 80 provoca rotações; depois remove-se 30.

Arvore Treap — Estado inicial (apos insercoes)



Arvore Treap — Estado intermediario (bifurcacao / rotacao / split)



Arvore Treap — Estado apos remocao / reorganizacao

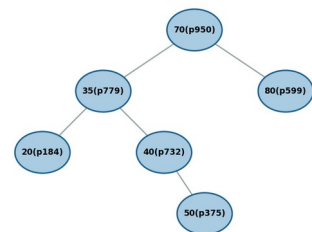


Figura 4 — Treap: BST pelas chaves e max-heap pelas prioridades; rotações na inserção e remoção.

3.5. KD-Tree

São inseridos seis pontos; o particionamento do plano mostra os cortes verticais (em x, nível par, em vermelho) e horizontais (em y, nível ímpar, em azul). A inserção de (60,50)

subdivide uma região; a remoção de $(30, 40)$ (a raiz) aciona a substituição pelo mínimo (findMin). A busca do vizinho mais próximo de $(55, 55)$ retorna corretamente $(60, 50)$.

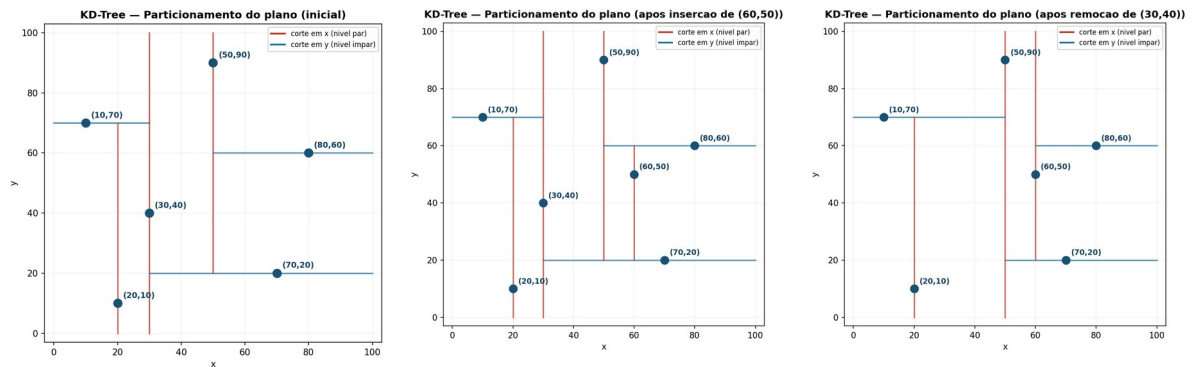


Figura 5 — KD-Tree: particionamento do plano nos três estados (inicial, após inserir $(60, 50)$ e após remover $(30, 40)$).

KD-Tree — Estado inicial (apos insercoes)

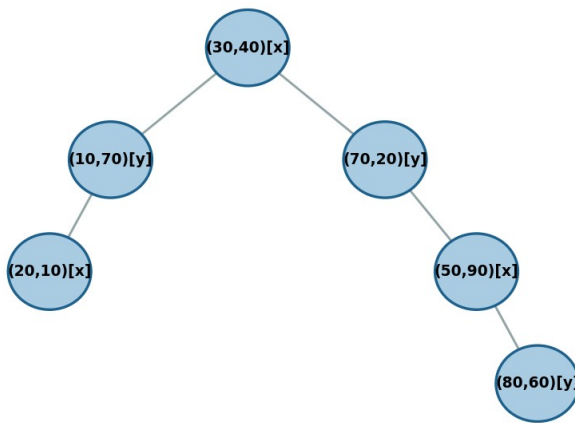


Figura 6 — Topologia da KD-Tree no estado inicial. Cada rótulo indica o ponto e a dimensão de corte $[x]$ ou $[y]$.

4. Análise de Complexidade e Comparação Teórica

A Tabela 1 resume a complexidade assintótica das operações. Notação: n = número de elementos; m = comprimento da chave (string); N = número de chaves; σ = tamanho do alfabeto. Quando há duas expressões separadas por barra, indicam **caso médio** / **pior caso**; ^a = amortizado; ^e = esperado (probabilístico); * = pior caso de espaço da Trie.

Estrutura	Busca	Inserção	Remoção	Espaço	Constr.
Trie	$O(m)$	$O(m)$	$O(m)$	$O(N \cdot m \cdot \sigma)^*$	$O(N \cdot m)$
Patricia	$O(m)$	$O(m)$	$O(m)$	$O(N)$	$O(N \cdot m)$
Splay	$O(1) / O(\log n)^a / O(n)$	$O(\log n)^a$	$O(\log n)^a$	$O(n)$	$O(n \log n)$
Treap	$O(\log n)^e / O(n)$	$O(\log n)^e$	$O(\log n)^e$	$O(n)$	$O(n \log n)$
KD-Tree	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n)$	$O(n \log n)$
BST (ref.)	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n)$	$O(n \log n)$
AVL (ref.)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n \log n)$

Tabela 1 — Complexidade assintótica das operações (com BST e AVL como referência).

4.1. Discussão dos custos

- **Trie e Patricia:** as operações custam $O(m)$ porque dependem apenas de percorrer os m caracteres da chave, e não de n . A Patricia executa menos passos por não visitar nós intermediários redundantes, e usa $O(N)$ nós contra o pior caso $O(N \cdot m \cdot \sigma)$ da Trie — daí sua vantagem de memória.
- **Splay:** uma operação isolada pode custar $O(n)$ no pior caso (árvore encadeada), mas o Teorema do Balanceamento Amortizado garante $O(\log n)$ amortizado por operação em qualquer sequência. Acessos repetidos à raiz custam $O(1)$.
- **Treap:** como as prioridades são aleatórias, a altura esperada é $O(\log n)$; o pior caso $O(n)$ existe, mas tem probabilidade desprezível. O custo é probabilístico, não amortizado.
- **KD-Tree:** a construção balanceada custa $O(n \log n)$ e a busca é $O(\log n)$ em média; contudo, em altas dimensões ou com inserções desfavoráveis, a poda perde eficácia e a busca tende a $O(n)$.
- **Comparação com BST/AVL:** a AVL garante $O(\log n)$ no pior caso para busca/inserção/remoção; a BST simples só o faz em média. Splay e Treap abrem mão da garantia estrita da AVL em troca de simplicidade (Treap) ou de adaptação ao padrão de acesso (Splay).

5. Metodologia Experimental e Resultados

Metodologia. Os experimentos foram implementados em `src/bench.cpp` e compilados com `g++ -O2 -std=c++17`. Os tempos são medidos com `std::chrono::high_resolution_clock` e representam a soma de todas as operações de cada fase, em milissegundos. As entradas são pseudoaleatórias com semente fixa. Para evitar que o compilador eliminasse laços de busca cujo resultado não é usado, os resultados são acumulados em uma variável volátil. Foram avaliados diferentes tamanhos de entrada e um padrão de acesso com localidade controlada.

5.1. Trie × Patricia (strings)

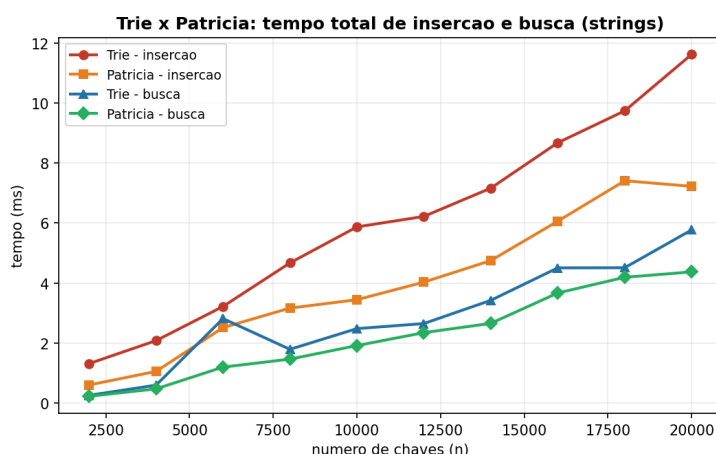


Figura 7 — Tempo total de inserção e busca de n strings aleatórias (comprimento 8).

A Patricia é consistentemente mais rápida que a Trie tanto na inserção quanto na busca, e cresce mais suavemente com n . O resultado confirma a teoria: ambas são $O(m)$, mas a compactação reduz o número de nós efetivamente visitados e o número de alocações, melhorando também o uso de memória e a localidade de cache.

5.2. Splay × Treap × BST balanceada (busca aleatória)

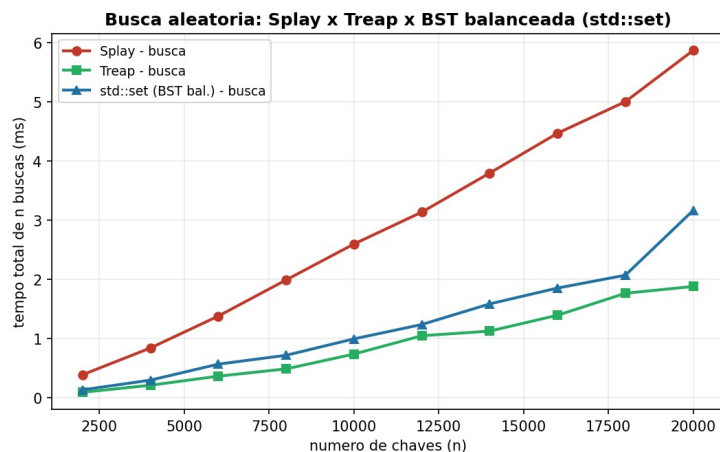


Figura 8 — Tempo total de n buscas aleatórias de inteiros (Splay, Treap e std::set).

Sob acesso puramente aleatório (sem localidade), a Splay é a mais lenta: cada busca realiza rotações (escritas em memória) mesmo quando não há ganho de localidade. A Treap e a `std::set` (uma BST balanceada, do tipo rubro-negra) apresentam desempenho próximo, coerente com o custo $O(\log n)$ de ambas. Esse é um caso em que o comportamento assintótico esperado se confirma na prática.

5.3. Efeito da localidade de acesso (vantagem da Splay)

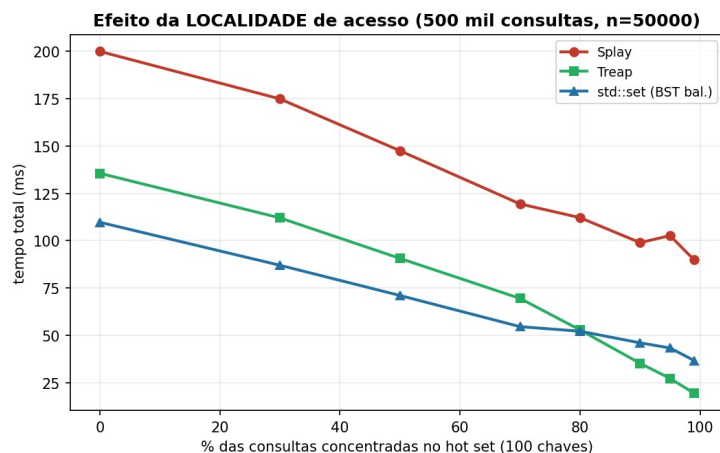


Figura 9 — 500 mil consultas em uma estrutura com $n = 50000$, variando a concentração no hot set (100 chaves).

Este experimento concentra progressivamente as consultas em um pequeno conjunto de 100 chaves. Todas as estruturas melhoram (efeito de cache), mas a Splay é a que mais se beneficia proporcionalmente: seu tempo cai de cerca de 200 ms (acesso uniforme) para cerca de 90 ms (99% concentrado), uma redução de ~55%. Isso ocorre porque as chaves quentes são levadas para perto da raiz e passam a ser encontradas em poucos passos. Ainda assim, em tempo absoluto ela permanece mais lenta que a BST balanceada, pois o custo constante das rotações não é compensado apenas pela menor profundidade. Trata-se de uma **divergência instrutiva** entre a análise assintótica (que favorece a Splay sob localidade) e o desempenho real (dominado por fatores constantes).

5.4. KD-Tree × busca linear (vizinho mais próximo)

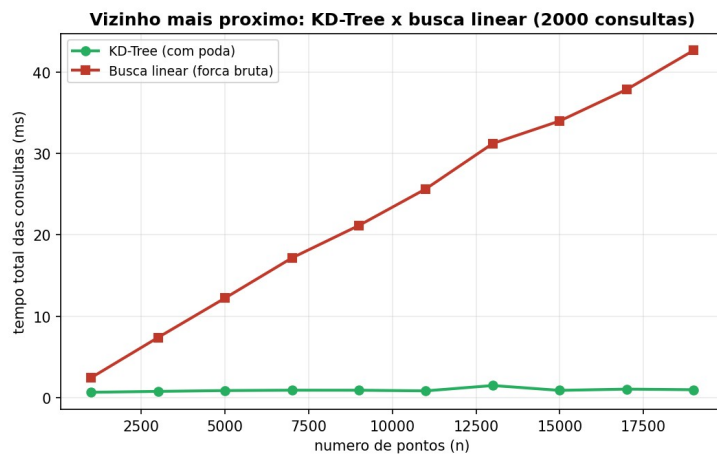


Figura 10 — Tempo de 2000 consultas de vizinho mais próximo: KD-Tree (com poda) × busca linear.

A busca linear cresce linearmente com n (cada consulta examina todos os pontos), enquanto a KD-Tree permanece praticamente constante graças à poda de regiões. Para $n = 19000$, a KD-Tree é cerca de $30\times$ mais rápida. Este é o caso de maior convergência entre teoria e prática: a vantagem $O(\log n)$ médio $\times O(n)$ aparece de forma nítida.

6. Aplicações, Análise Crítica e Discussão

- **Trie:** autocompletar, dicionários, corretores ortográficos e busca por prefixos — situações em que a chave é uma string e o compartilhamento de prefixos é frequente.
- **Patricia:** indexação compacta, tabelas de roteamento IP e índices baseados em prefixos, onde a economia de memória da compactação é decisiva.
- **Splay:** caches, montadores e sistemas com acessos altamente desiguais ou localidade temporal, em que itens recentes tendem a ser reaccessados.
- **Treap:** estruturas de busca de propósito geral que exigem balanceamento com implementação simples, e como base de estruturas mais avançadas (ex.: treaps implícitas para sequências).
- **KD-Tree:** busca espacial, sistemas GIS, computação gráfica, reconhecimento de padrões e algoritmos de aprendizado de máquina baseados em vizinhos (k-NN).

6.1. Vantagens, limitações e dificuldades de implementação

A **Trie** é simples, mas pode consumir muita memória; a **Patricia** corrige isso, ao custo de uma implementação mais delicada (split e merge foram as partes mais trabalhosas). A **Splay** tem código de rotação relativamente simples, mas depende de ponteiros de pai e sua vantagem só aparece sob localidade. A **Treap** foi a de implementação mais direta (recursão com rotações), com a ressalva de depender de um bom gerador de aleatoriedade. A **KD-Tree** tem inserção/busca simples, mas a remoção (via findMin alternando dimensões) foi a operação mais sutil de todo o trabalho.

Diferenças teórico × experimental. O caso mais marcante foi a Splay: teoricamente favorecida sob localidade, na prática permaneceu mais lenta em tempo absoluto por causa do custo constante das rotações — embora a tendência prevista (melhora com a localidade) tenha se confirmado. Nos demais casos, teoria e experimento convergiram bem, com destaque para a KD-Tree × busca linear.

Possíveis melhorias. Usar vetores de 26 posições na Trie (em vez de mapa) para acelerar o acesso; construir a KD-Tree de forma balanceada pela mediana; implementar a Splay em

versão top-down para reduzir constantes; e adicionar treaps implícitas para operações de sequência.

7. Conclusão

O trabalho respondeu, de forma fundamentada, às questões norteadoras:

1. **Principais diferenças:** as cinco estruturas diferem no tipo de chave (string, escalar, ponto) e na estratégia de organização (compartilhamento de prefixos, autoajuste por acesso, aleatoriedade, particionamento espacial).
2. **Melhor desempenho por operação/dados:** Patricia para strings; BST balanceada/Treap para busca aleatória de inteiros; Splay para acessos com forte localidade; KD-Tree para busca espacial/vizinho mais próximo.
3. **Coerência com a complexidade:** em geral sim — sobretudo na KD-Tree $\times$ busca linear e na Trie $\times$ Patricia. A exceção instrutiva foi a Splay, cujos fatores constantes mascararam parte da vantagem assintótica.
4. **Maior dificuldade de implementação:** a recompactação (merge) da Patricia e a remoção da KD-Tree, por exigirem cuidado com casos de borda.
5. **Cenários ideais:** cada estrutura tem seu nicho, conforme discutido na Seção 6 — não há vencedora universal.
6. **Vantagens e limitações:** resumidas na Seção 6.1; a escolha correta depende do padrão de dados e de acesso do problema.

Em síntese, o estudo evidenciou a relação direta entre as **propriedades estruturais**, a **complexidade algorítmica** e o **desempenho observado**. Compreender essas relações é o que permite justificar tecnicamente a escolha de uma estrutura para um problema computacional específico — objetivo central deste trabalho.

Referências

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.
- SEdgeWICK, R.; WAYNE, K. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.
- SLEATOR, D. D.; TARJAN, R. E. Self-Adjusting Binary Search Trees. *Journal of the ACM*, v. 32, n. 3, p. 652–686, 1985.
- ARAGON, C. R.; SEIDEL, R. Randomized Search Trees. In: *Proc. 30th FOCS*, 1989, p. 540–545.
- BENTLEY, J. L. Multidimensional Binary Search Trees Used for Associative Searching. *Communications of the ACM*, v. 18, n. 9, p. 509–517, 1975.
- MORRISON, D. R. PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. *Journal of the ACM*, v. 15, n. 4, p. 514–534, 1968.